PROJECT 3 — CS280A

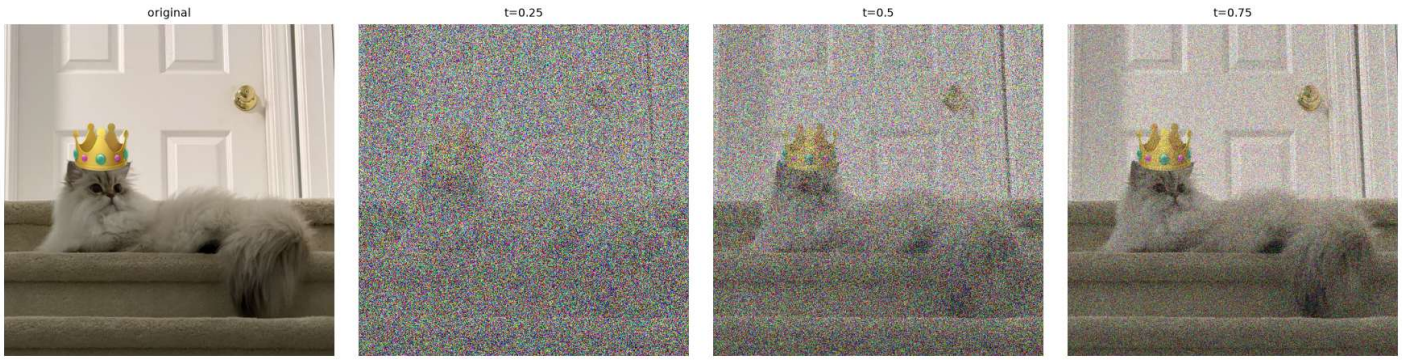
Flow Models: Exploring and Building Them

Part A plays with a big pretrained text-to-image flow model (PixNerd-XXL). Part B builds a flow-matching model for MNIST from scratch.

Part A: Exploring the Power of Flow Models

A · 1.1 Constructing Noisy Training Inputs

I sampled **one** noise tensor and reused it for t = 0.25, 0.5 and 0.75 on my own photo (my cat, wearing a crown). The function returns exactly the noise at t = 0 and the image at t = 1.



original, then x_t at t = 0.25 / 0.50 / 0.75 (same ε)

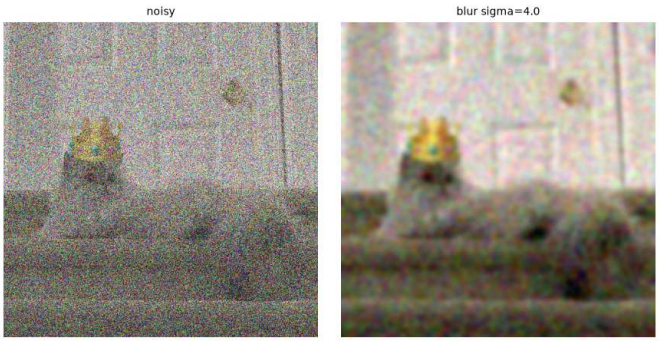
At t = 0.25 only the brightest, largest structures survive (the yellow crown and the white door are faint blobs). By t = 0.5 the cat is recognizable, and t = 0.75 is mostly clean with grain on top.

A · 1.2 Classical Denoising

Baseline: undo the amplitude scaling with x_t/t, then Gaussian blur. I swept $\sigma \in \{1, 2, 4, 8, 16\}$ and kept the σ with the lowest MSE to the clean image.



t = 0.25 → best σ = 16



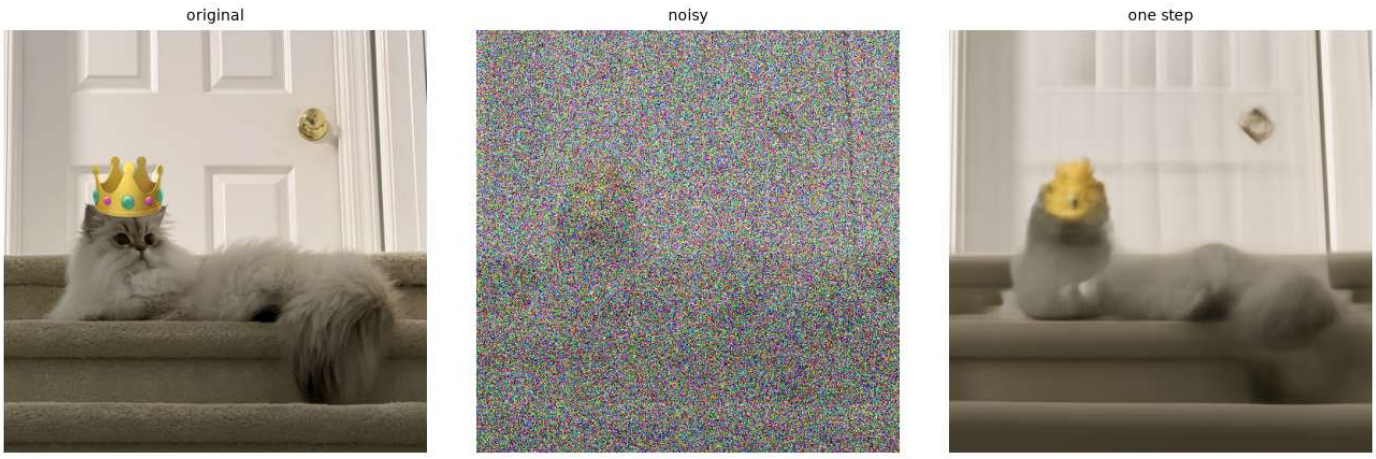
t = 0.50 → best σ = 4



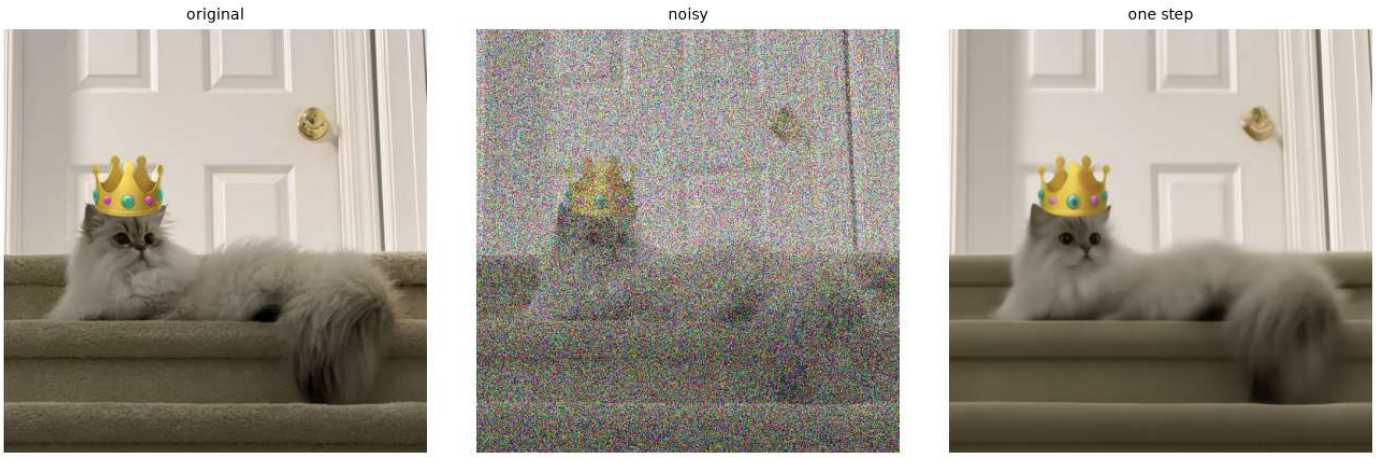
t = 0.75 → best σ = 2

A · 1.3 One-Step Denoising

One velocity prediction is enough to jump straight to an estimate of the clean image, $\hat{x}_1 = x_t + (1 - t)\, v_\theta(x_t, t, c)$, with c = "a high quality photo".



t = 0.25: original / noisy / one-step estimate



t = 0.50



t = 0.75

A · 1.4 Visualizing Velocity

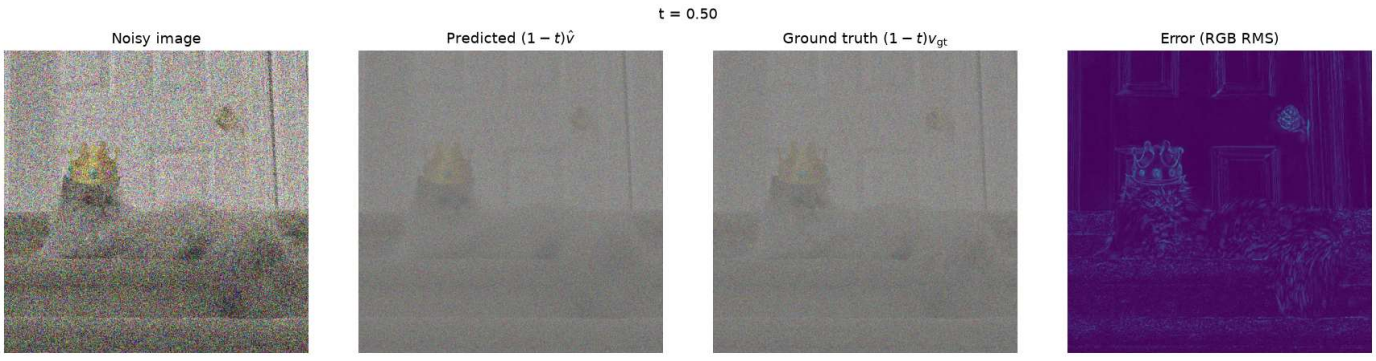
Here I compare what the model ***adds*** in one-step denoising, $(1 - t)\,\hat{v}$, with the update that would recover my original exactly, $(1 - t)(x_1 - \varepsilon) = x_1 - x_t$. The error map is the per-pixel RMS over RGB of their difference. Updates share one symmetric colour scale (0 = mid gray) and the error maps share one vmin/vmax across all three times.



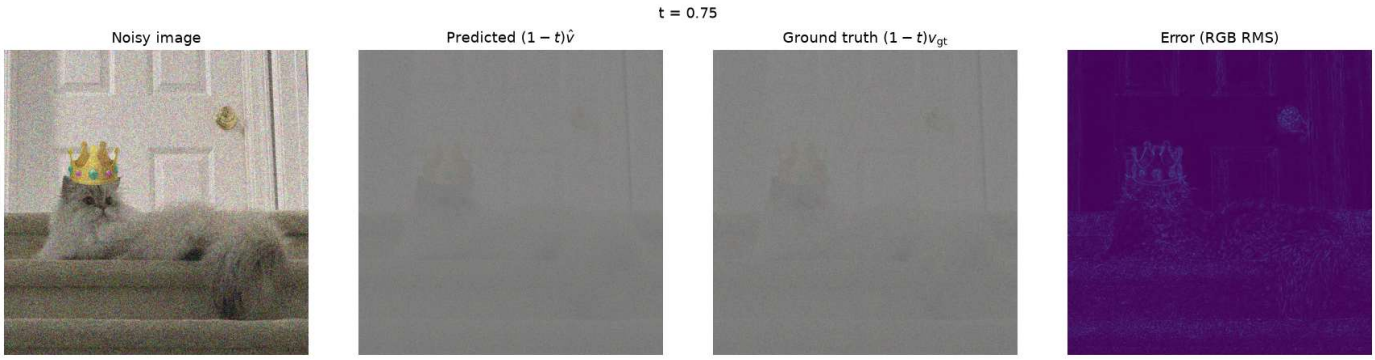
original x_1



t = 0.25: noisy · predicted update · ground-truth update · error



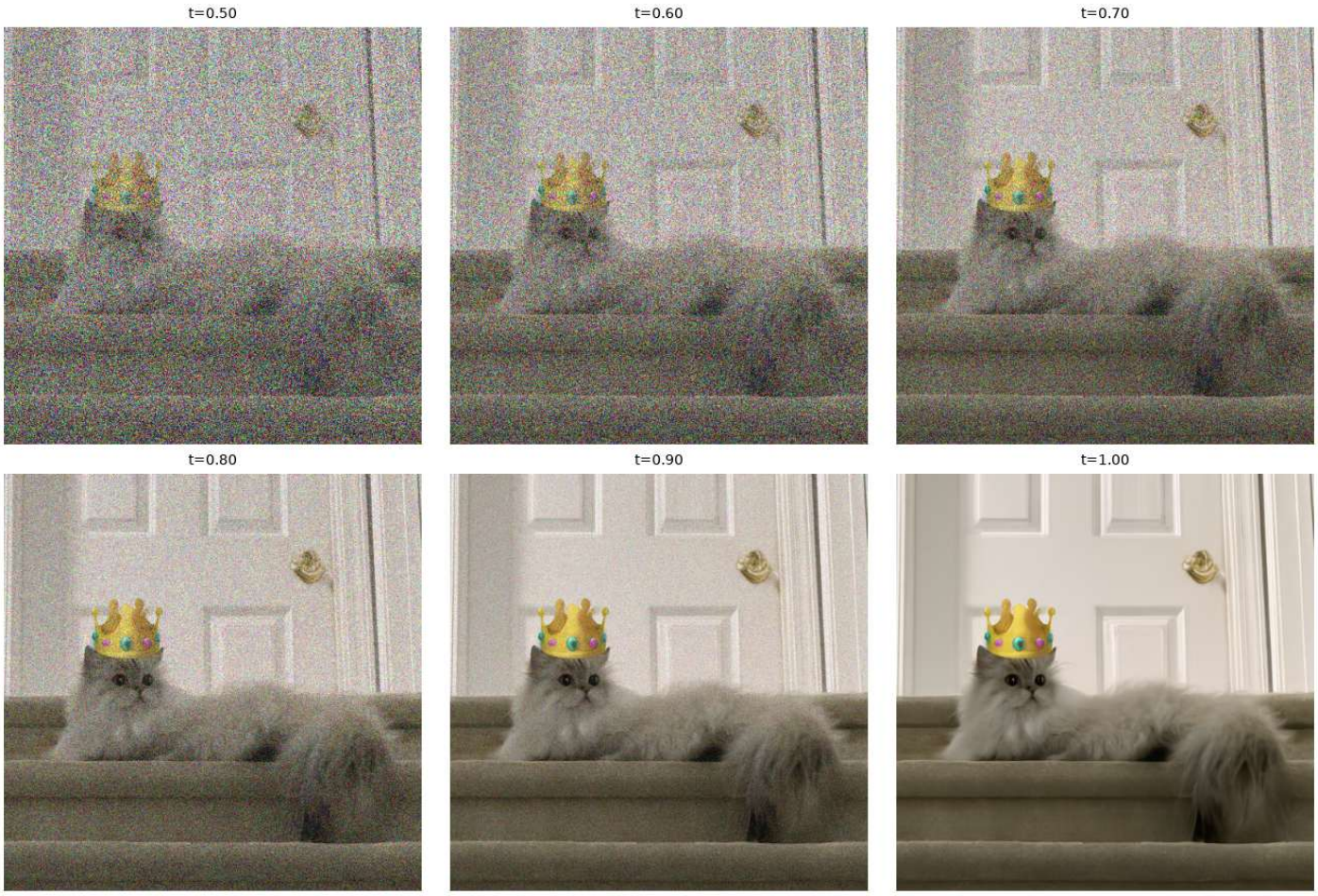
t = 0.50



t = 0.75

A · 1.5 Iterative Denoising with Euler Sampling

Instead of one big jump, Euler integration takes many small steps $x_{t+h} = x_t + h \cdot v(x_t, t)$, re-evaluating the velocity every time. The last step is shortened so the result lands exactly on t = 1. From t = 0.5 with h = 0.02 that's 25 model calls. I save the initial state, every 5th update and the final state. The notebook's checks pass (50 / 25 / 4 updates from t = 0 / 0.5 / 0.93, ending at 1).



trajectory from the t = 0.5 input: t = 0.50, 0.60, ..., 1.00



same t = 0.5 input: original / best blur / one step / Euler

Reconstruction MSE to the original (t = 0.5 input, pixels in [−1, 1]).

METHOD	MSE	LOOKS
Gaussian blur (σ = 4)	`0.01150`	blurry, coloured speckle
one-step denoising	**`0.00282`**	**clean but soft (fur and carpet smoothed)**
Euler, 25 steps	`0.00343`	sharpest: real fur and carpet texture

Why Euler looks best but doesn't win on MSE

- In the trajectory the noise fades steadily and the details sharpen late: the fur and the carpet only appear in the last couple of saved states.
- One-step denoising outputs something close to the *mean* of all plausible clean images, and the mean is exactly what minimizes MSE. That's why it is soft but scores best.
- Euler commits to *one* sharp, plausible sample. Its fur strands and door highlights are realistic, but they don't match my photo pixel-for-pixel, so MSE goes up slightly even though it looks much better.

A · 1.6 Generating Images from Noise

Starting at t = 0 from pure noise and running all 50 Euler steps with "a high quality photo" (no guidance), with seeds 180–184:



seed 180



seed 181



seed 182



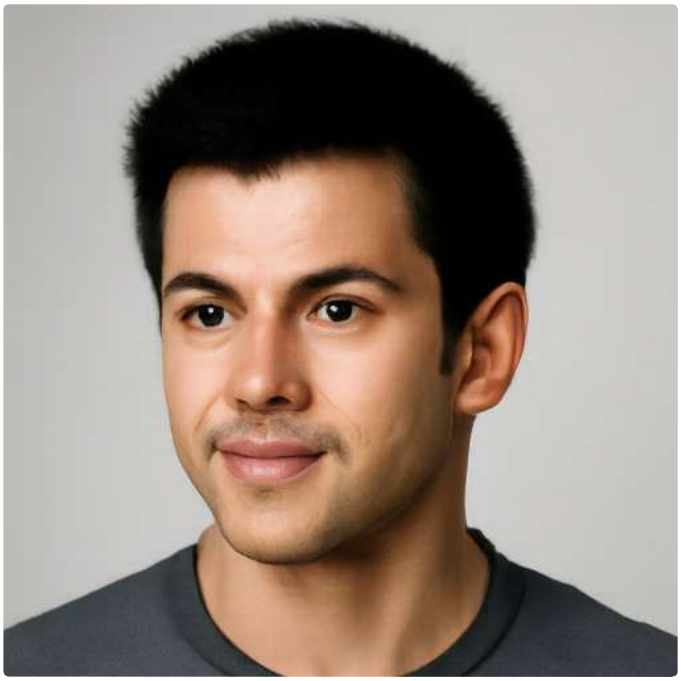
seed 183



seed 184

A · 2.1 Classifier-Free Guidance

CFG pushes the velocity away from the unconditional prediction (empty prompt "") and toward the conditional one, evaluated at the same x and t. With w = 0 it reduces to the conditional prediction, which the notebook check confirms. Same five seeds as above, w = 7:



seed 180, w = 7



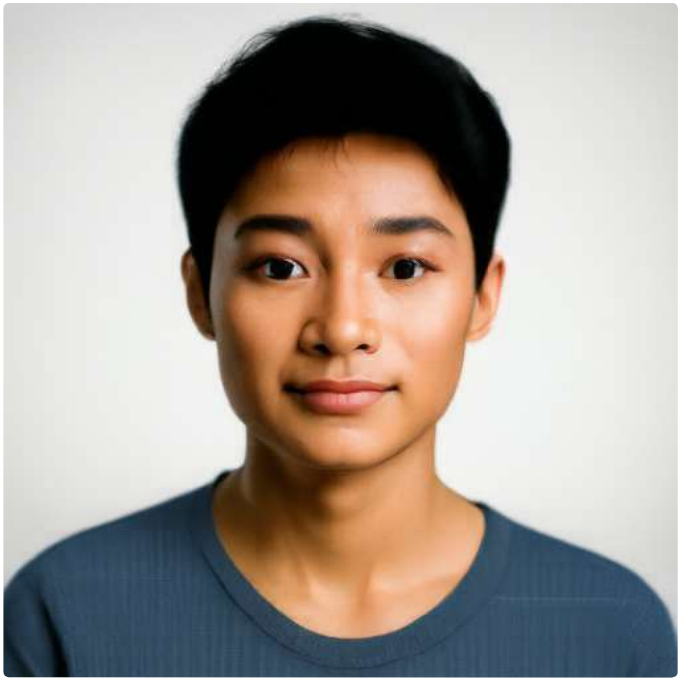
seed 181, w = 7



seed 182, w = 7



seed 183, w = 7



seed 184, w = 7

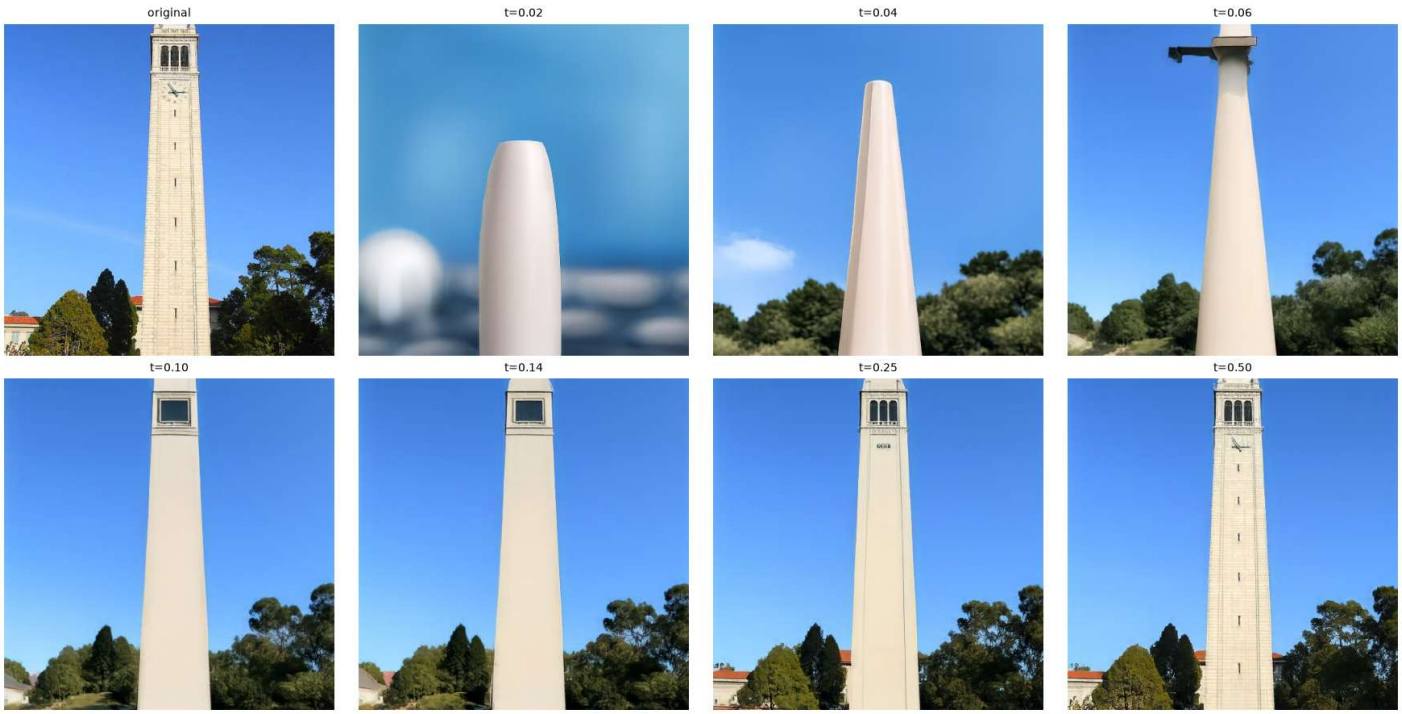
Comparison with 1.6

- **Much higher quality:** every sample is now a sharp, well-lit, anatomically correct face. The broken faces from seeds 180 and 183 are fixed.
- **Same noise, similar layout:** each image keeps some of its 1.6 counterpart. Seed 182 still has the hand against the cheek, and the light/dark backgrounds mostly carry over.
- **Less diversity:** all five are now centered, front-facing "stock photo" headshots with higher contrast and more saturated skin. Guidance trades variety for prompt fidelity and polish.

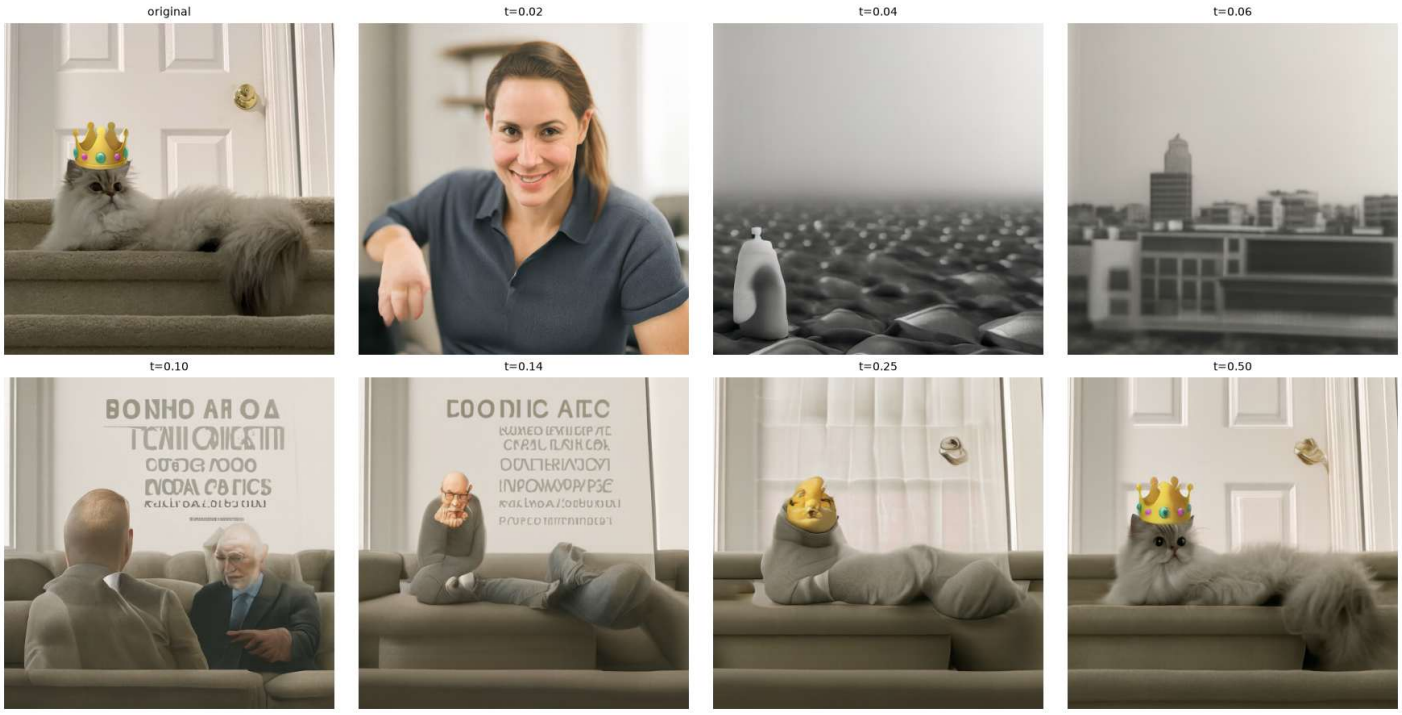
A · 3.1 Image-to-Image Editing (SDEdit)

Noise a real image to a start time t_{start}, then integrate to t = 1 with CFG and "a high quality photo". A small t_{start} means mostly noise, so the model is free to invent. A large t_{start} keeps most of the image. All seven panels use the same noise (seed 180).

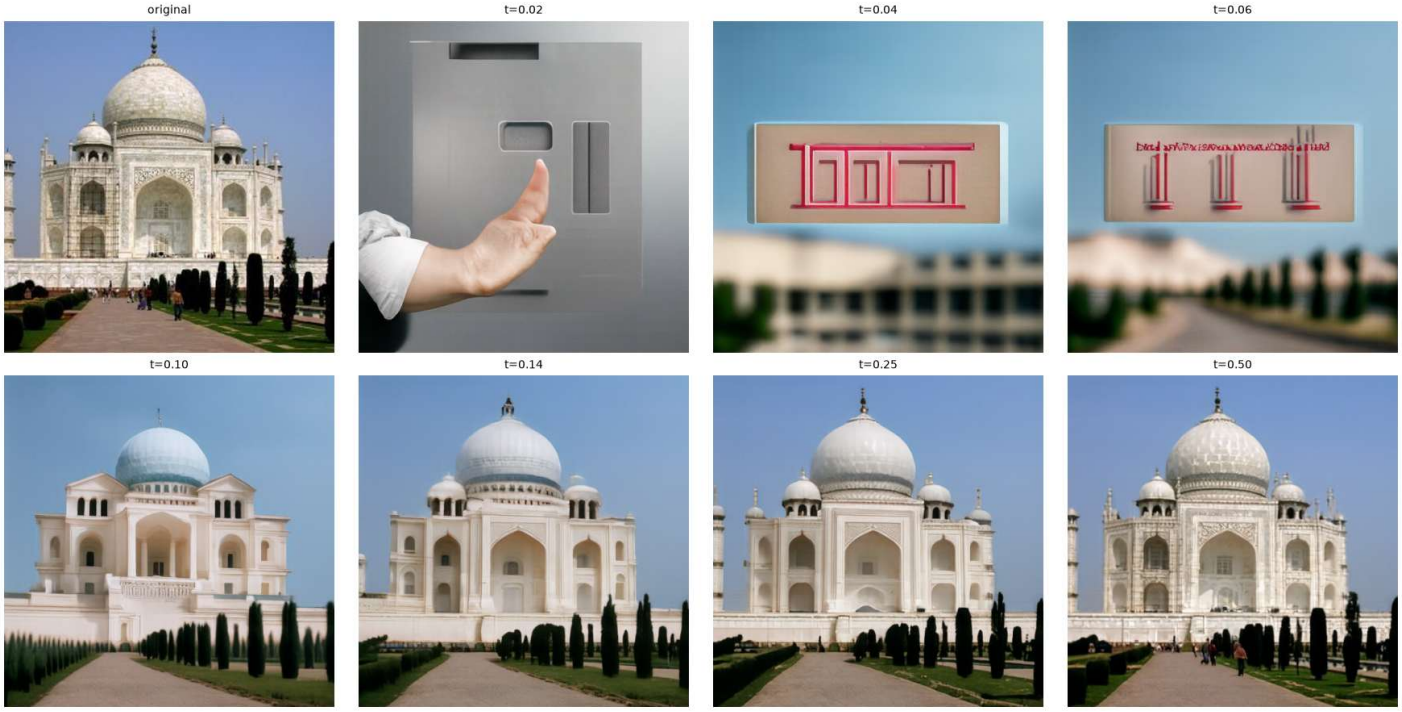
CAMPANILE AND TWO OF MY OWN PHOTOS



Campanile — t_{start} = 0.02, 0.04, 0.06, 0.10, 0.14, 0.25, 0.50



my cat

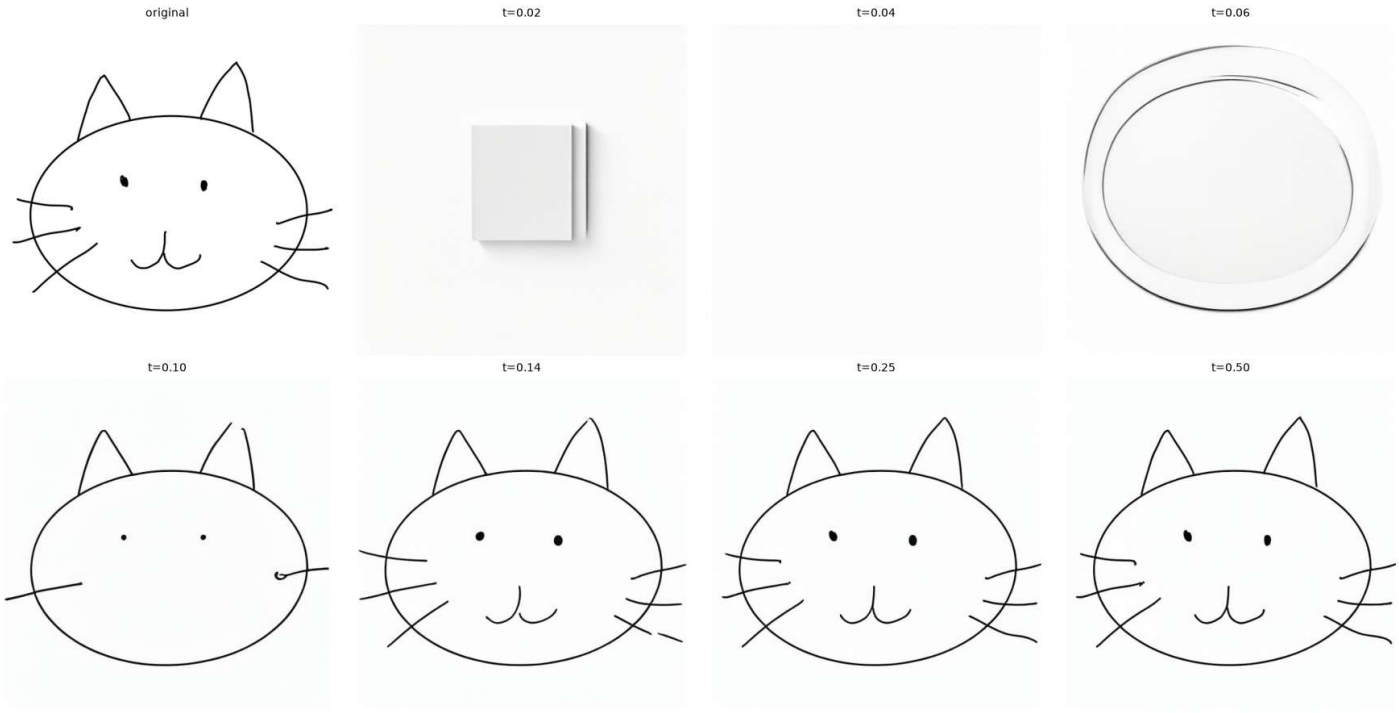


Taj Mahal (my photo)

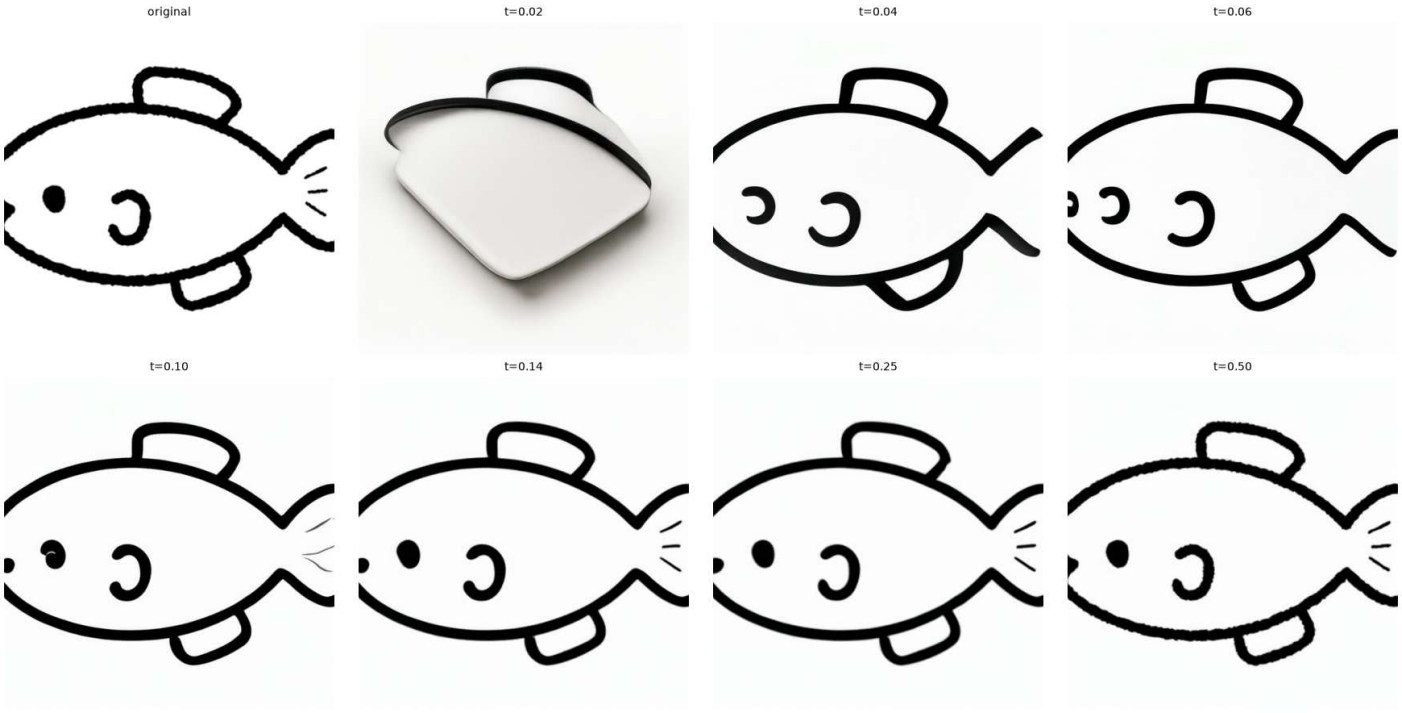
ONE WEB IMAGE AND TWO HAND DRAWINGS



web image: engraving of a hand



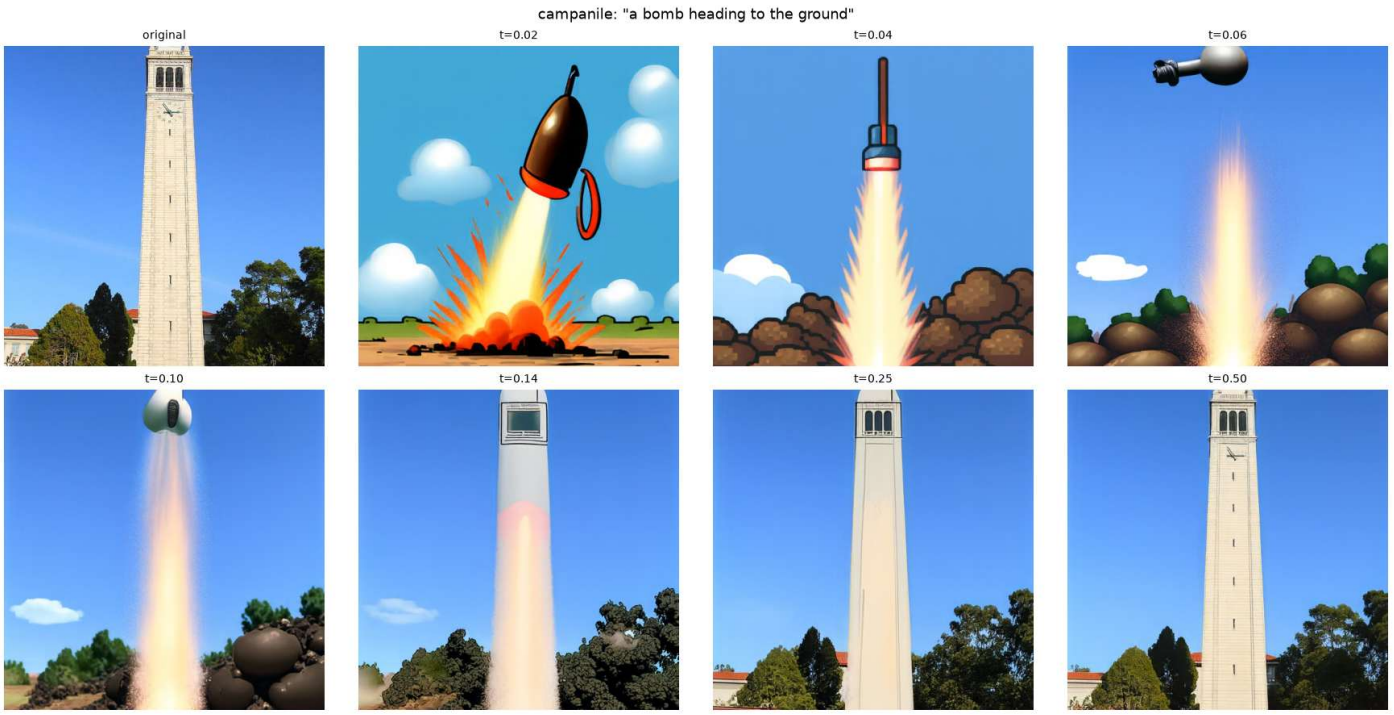
my drawing 1: cat doodle



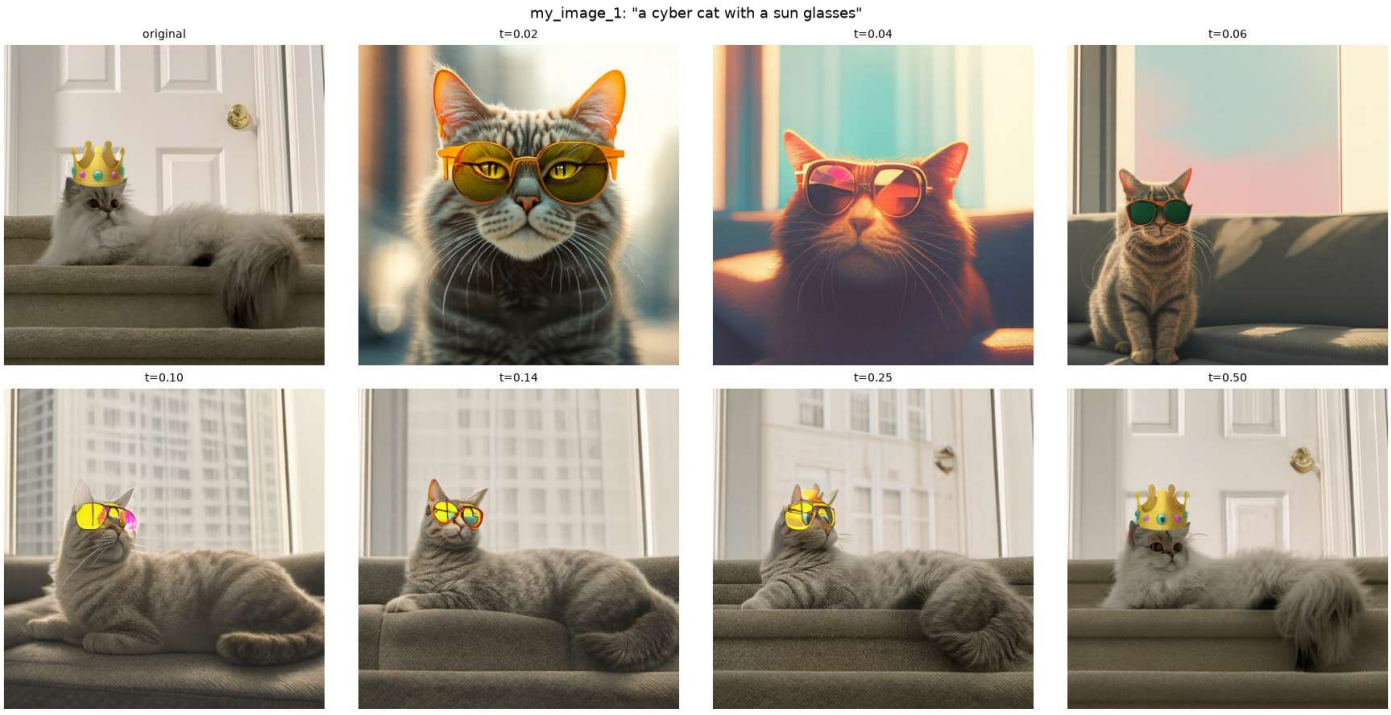
my drawing 2: fish doodle

A · 3.2 Text-Guided Image Editing

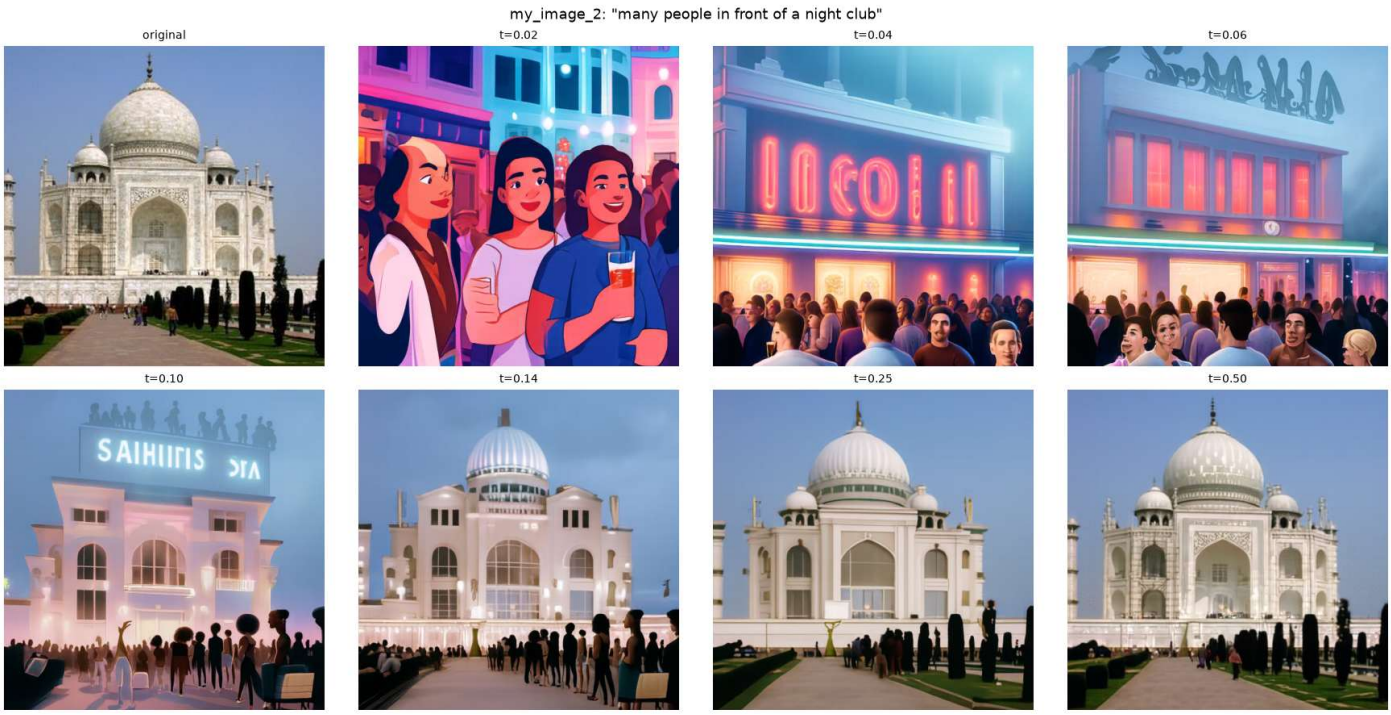
Same procedure and the same seven start times, but each image gets its own prompt in place of "a high quality photo".



Campanile — *"a bomb heading to the ground"*



my cat — *"a cyber cat with a sun glasses"*



Taj Mahal — *"many people in front of a night club"*

A · 3.3 Visual Anagrams

One image that shows prompt a upright and prompt b upside down. At every Euler step I average the CFG velocity for prompt a with the velocity for prompt b computed on the ***rotated*** image and rotated back. F is a 180° rotation (both spatial axes flipped).

an oil painting of a dog



an oil painting of a bowl of fruit



upright: "an oil painting of a dog"
rotated 180°: "an oil painting of a bowl of fruit"

A watercolor of a fish



A watercolor of a bird



upright: "a watercolor of a fish"
rotated 180°: "a watercolor of a bird" (failed)

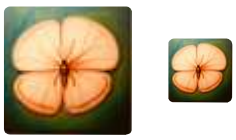
A · 3.4 Hybrid Images

Factorized diffusion: take the low frequencies of prompt a's velocity and the high frequencies of prompt b's, at the same x and t. The low-pass is a Gaussian blur, kernel 265 and σ = 16 at 512×512. Far away (or

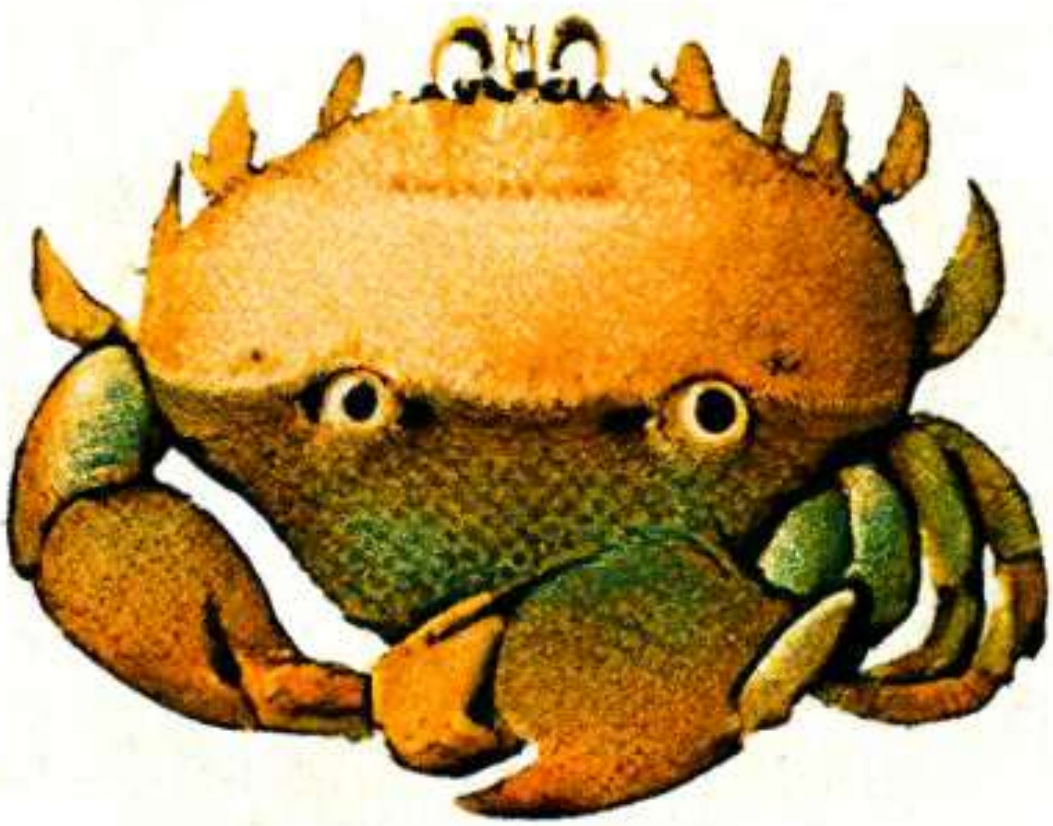
small) you see prompt a; up close you see prompt b.



full size — close up: "an oil painting of a butterfly"



reduced (64 px, 32 px) — from far: "an oil painting of a flower"



full size — close up: "a lithograph of a crab"



reduced — from far: "a lithograph of a hamburger"

A · BELLS & WHISTLES

Visual Anagrams with Two More Transformations

Following Visual Anagrams (Geng et al.), the 3.3 trick works for any view A that is an **orthogonal** linear map of the pixels: you average each view's velocity after mapping it back with A^{-1}. I added two views from the paper:

- **90° rotation** (clockwise): a global pixel permutation.
- **Inner-circle rotation:** only a central disk (60% of the width) rotates by 90°, and the outer ring stays put. The disk is measured from the exact grid centre (255.5, 255.5), so it maps onto itself under a 90° rotation and the view is an exact permutation.



upright: "an oil painting of a rabbit"
rotated 90° clockwise: "an oil painting of a duck" (seed 181)



upright: "an oil painting of an owl"
inner circle rotated 90°: "an oil painting of a bowl of fruit" (seed 180)

A · BELLS & WHISTLES

Course Logo

Text-guided image-to-image (SDEdit with a prompt, the `edit` function from 3.2) starting from the UC Berkeley seal. Logos often have transparent backgrounds, so I composited it on white and padded it to a square before preprocessing so nothing gets cropped.



first try, $t_{start} = 0.25$

"a minimalist flat vector logo of a bear looking through a camera lens, navy blue and gold"; 0.25–0.60 all looked like this



final logo, prompt 3, $t_{start} = 0.10$

"an embroidered circular patch with a golden bear in the center, navy blue and gold thread"



prompt 1 (bear with camera) at $t_{start} = 0.06 / 0.10 / 0.14 / 0.20$



prompt 2: "a circular navy blue and gold badge with a camera lens in the center, minimalist vector logo"

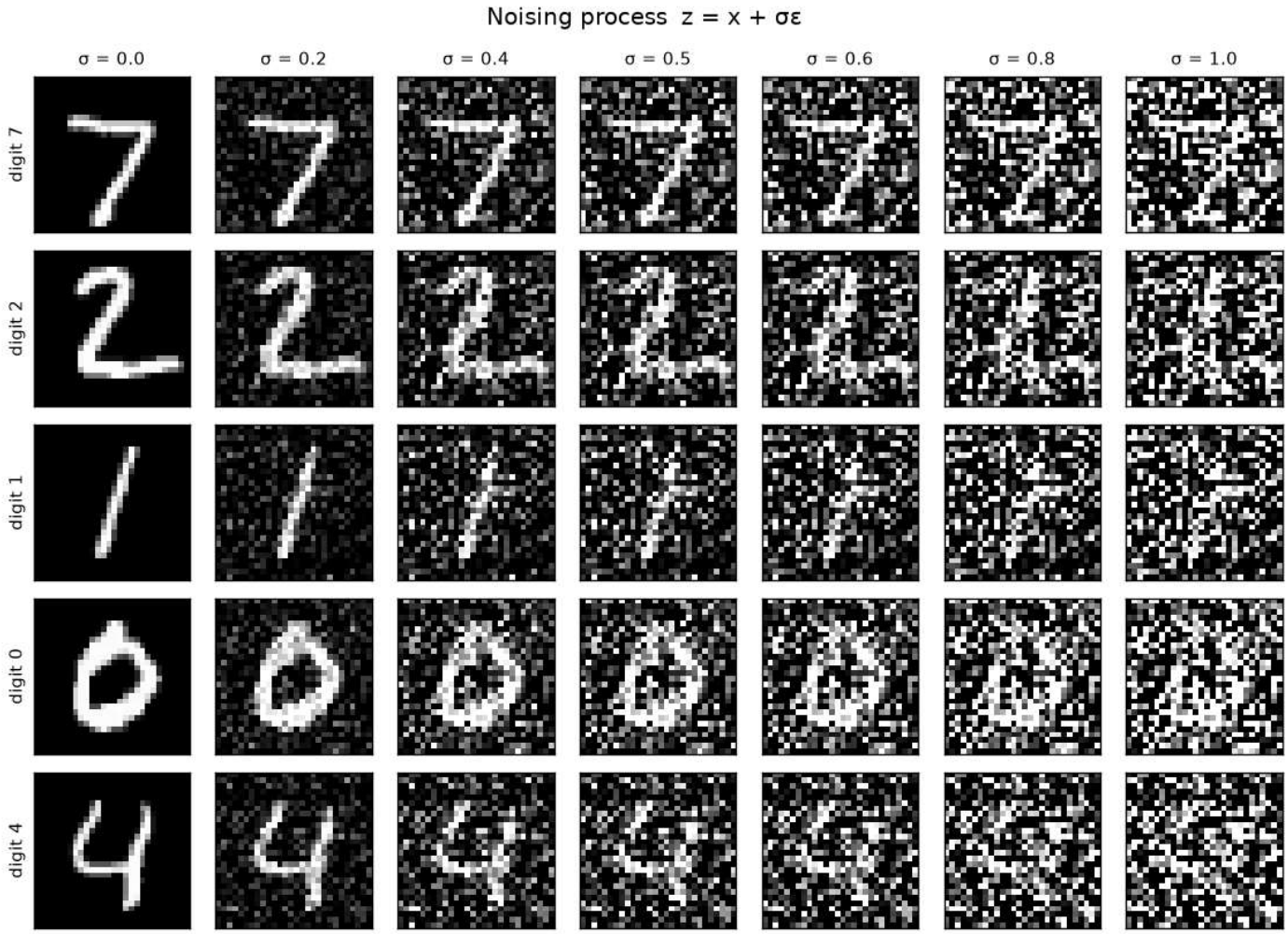


prompt 3: "an embroidered circular patch with a golden bear in the center, navy blue and gold thread"

Part B: Flow Matching from Scratch

B · 1.2 Using the UNet to Train a Denoiser

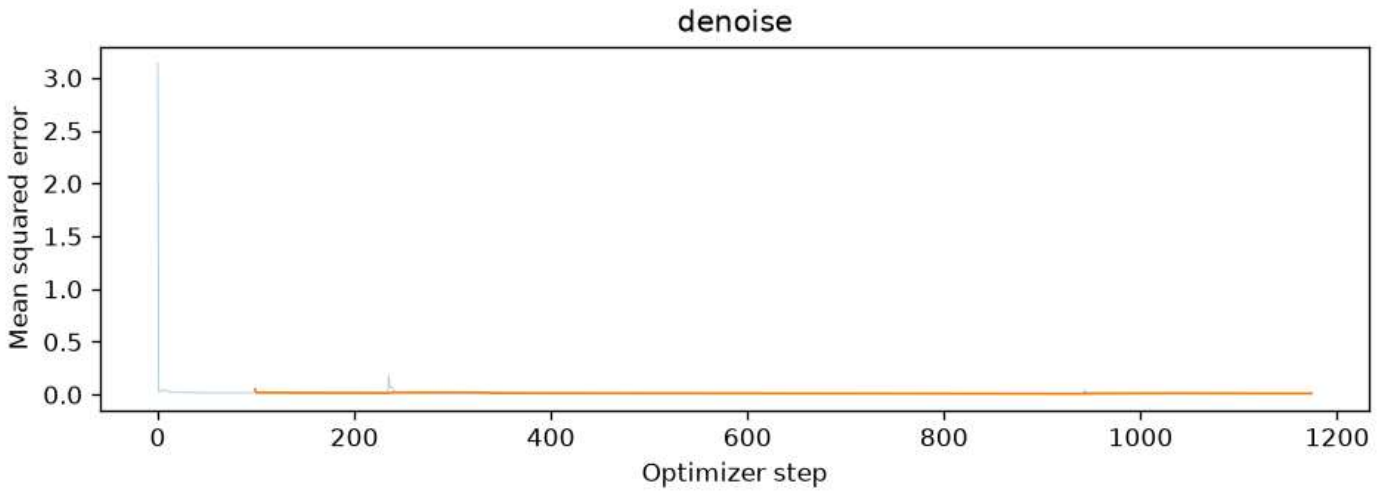
Noising process z = x + σε. Each row below is one digit at σ = 0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0:



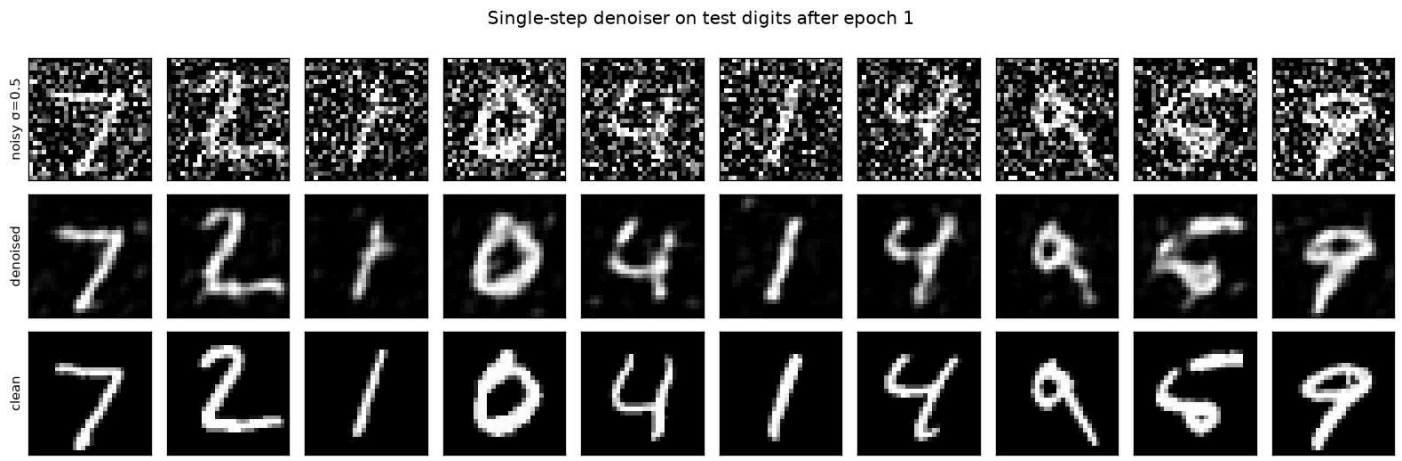
noising process, σ from 0 to 1

1.2.1 Training

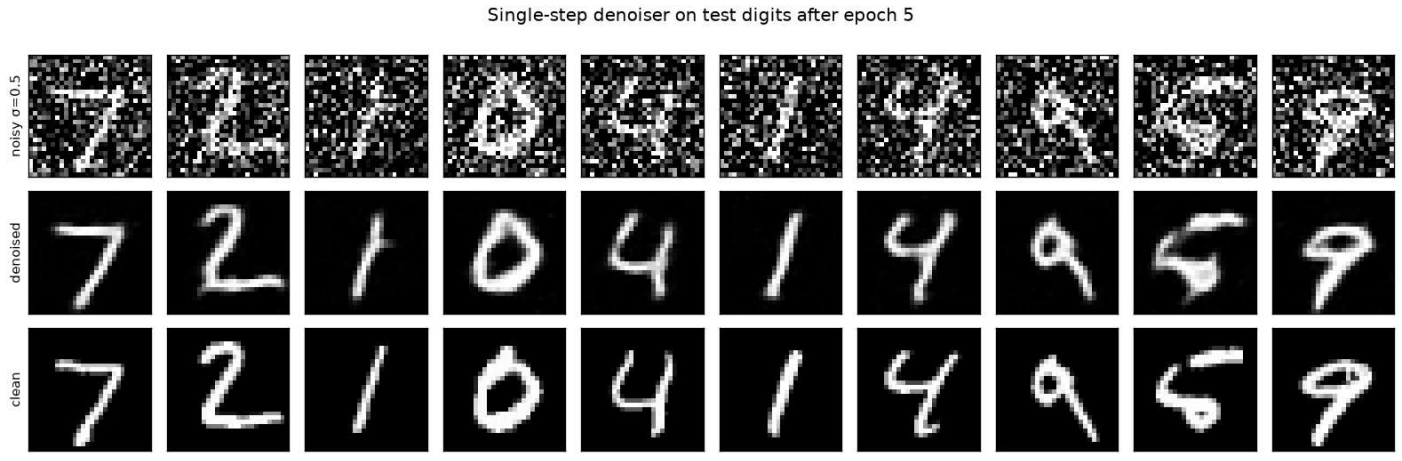
Trained to map z (σ = 0.5) back to x with an L2 loss. Batch 256, 5 epochs, D = 128, Adam lr 1e-4, shuffled training set, and fresh noise drawn every time a batch is fetched. Mean loss per epoch: 0.0313 → 0.0161 → 0.0111 → 0.0093 → 0.0121. The last epoch bumps up a bit, which is ordinary minibatch noise at this lr; the samples still improve.



training loss ($\sigma = 0.5$)



test set after epoch 1: noisy / denoised / clean

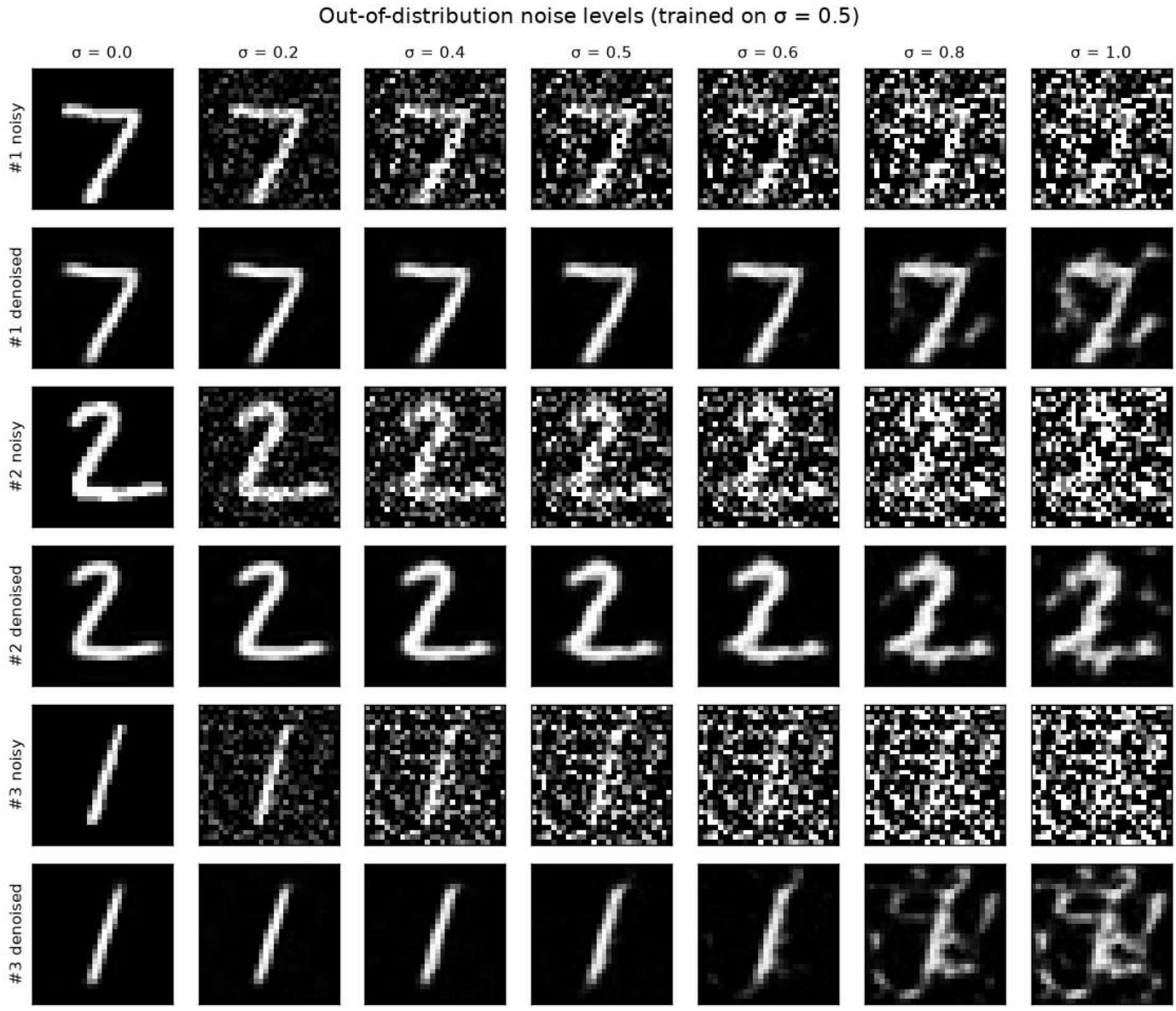


after epoch 5

After epoch 1 the digits are already readable but blotchy, with background speckle. After epoch 5 strokes are clean and the background is black. Some ambiguous inputs (the "5" in column 9) come out as a smudge,

since the noise destroyed too much to tell.

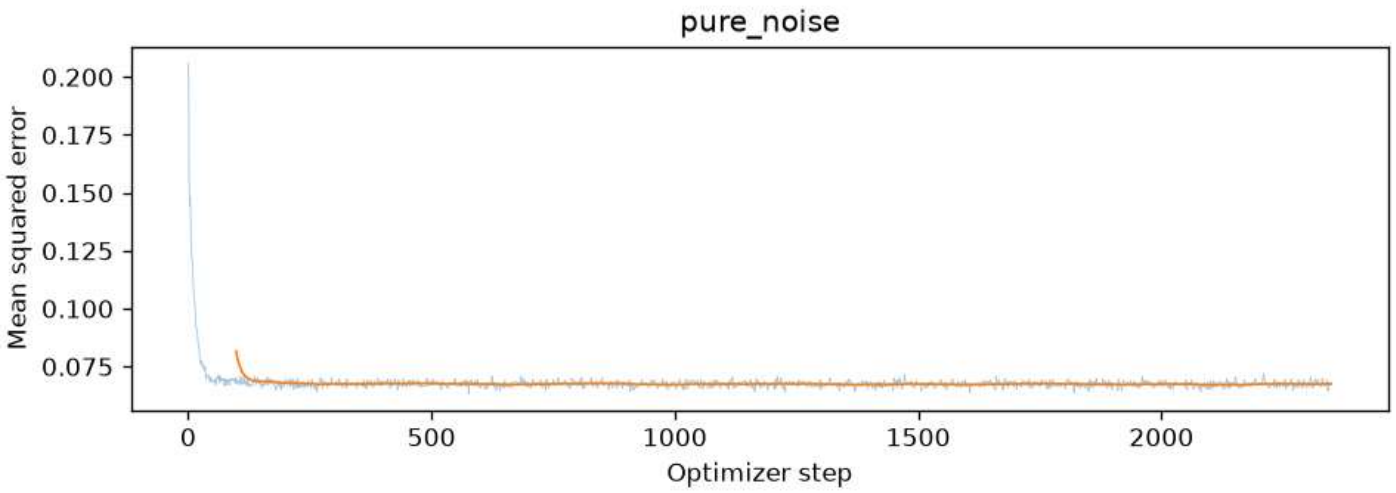
1.2.2 Out-of-Distribution Testing



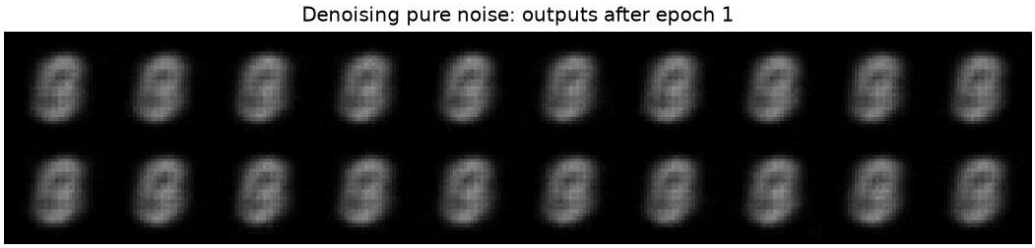
same three test digits, σ = 0 … 1.0, denoiser trained only on σ = 0.5

B · 1.2.3 Denoising Pure Noise

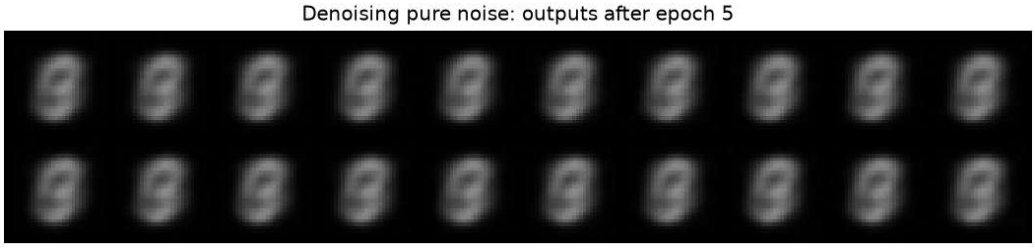
Same setup, but the input is pure noise z = ε and the target is still a clean digit. The loss flat-lines at about 0.0675 for all 5 epochs.



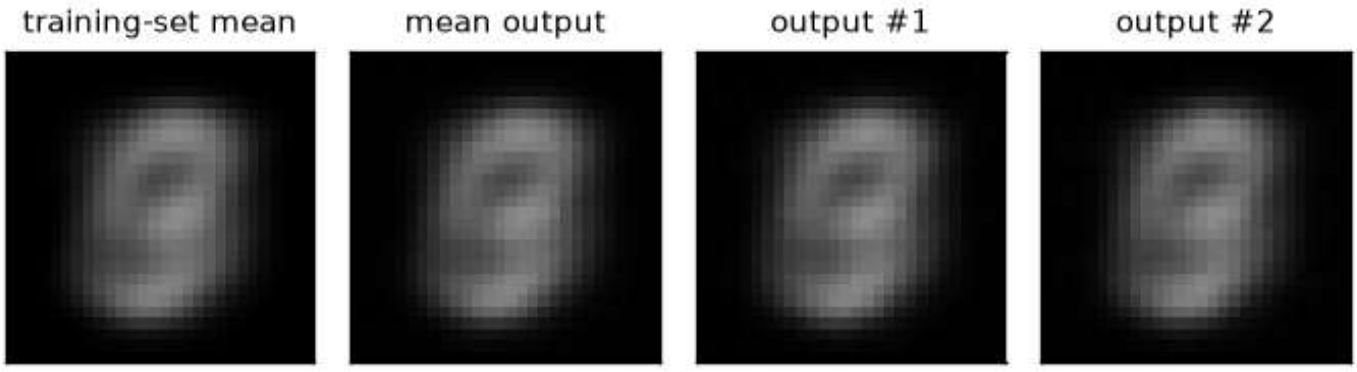
training loss



outputs for 20 different noise inputs, epoch 1



epoch 5



output vs. the average training digit

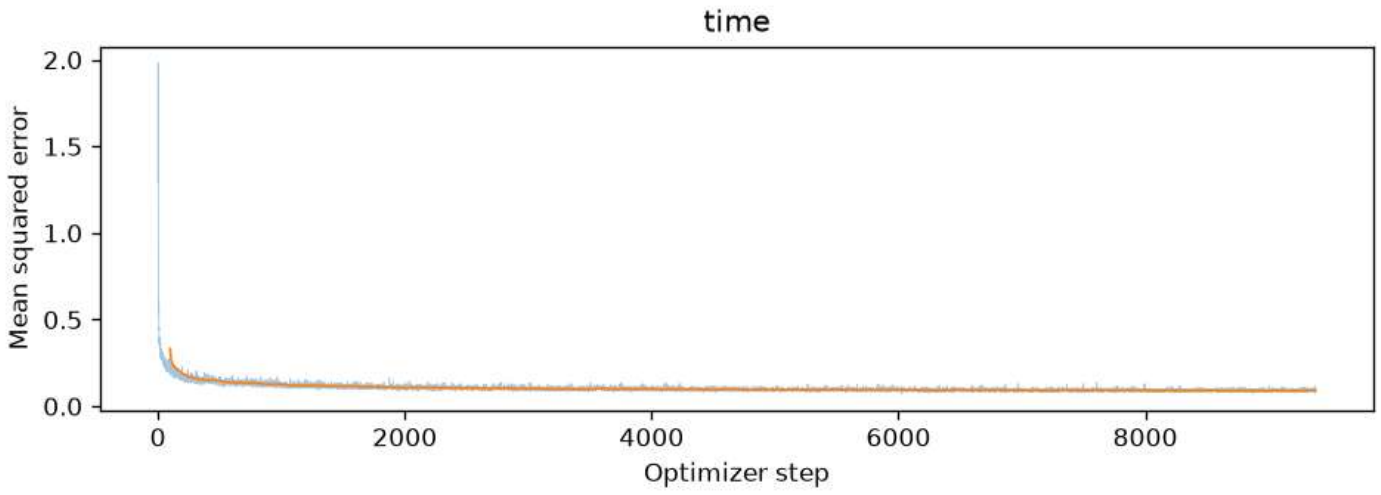
What the pattern is, and why

- **Every output is the same blurry ghost digit**, a fuzzy oval with hints of 0, 3, 8 and 9 overlaid, no matter which noise goes in. It matches the **mean of the training set**: the largest pixel difference is 0.037 and the outputs vary by only 0.004 (std) across inputs.
- Under an MSE loss the best possible prediction is the conditional expectation E[x | z]. Here z is pure noise, *independent* of x, so E[x | z] = E[x]: the average digit. The network learns to ignore its input and output that constant. That's also why the loss can't go below ≈ 0.0675, which is the pixel variance of MNIST around its mean.

B · 2.1–2.3

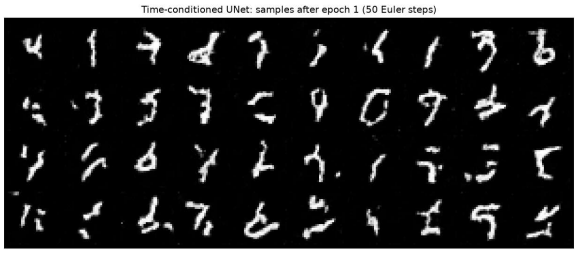
Time-Conditioned Flow Matching

Training: batch 64, D = 64, Adam lr 1e-2 with ExponentialLR $\gamma = 0.1^{1/10}$ (stepped once per epoch, 1e-2 → 1e-3), 10 epochs, with noise and t drawn fresh for every batch. Mean loss per epoch fell from 0.166 to 0.091.

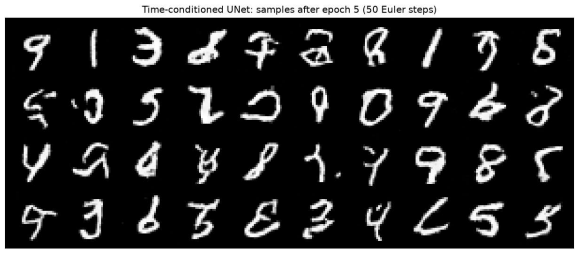


training loss, whole run

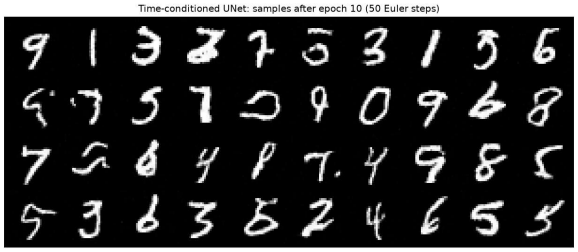
SAMPLES, 50 EULER STEPS, SAME SEED EACH TIME



epoch 1



epoch 5



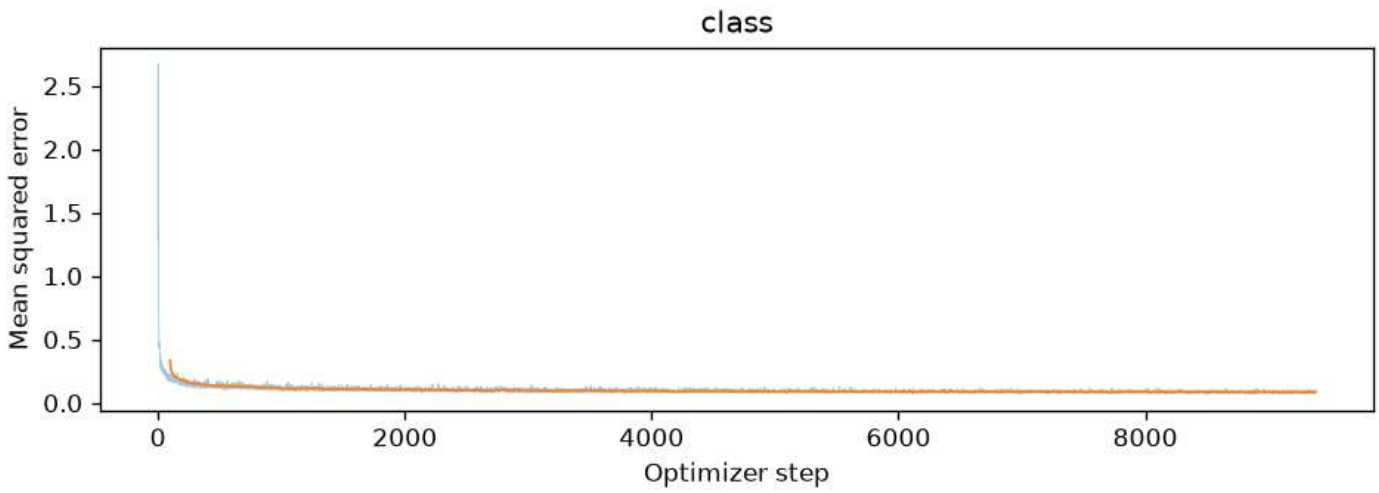
epoch 10

After epoch 1 the samples are thin, broken strokes. By epoch 10 most are legible digits (9, 1, 3, 0, 6, 8, 5…), but some are still malformed (a "7." with a stray dot, a squiggle in row 2). The model only knows the time, not which digit it's drawing, and it has no way to commit early.

B · 2.4–2.6 Class-Conditioned Flow Matching

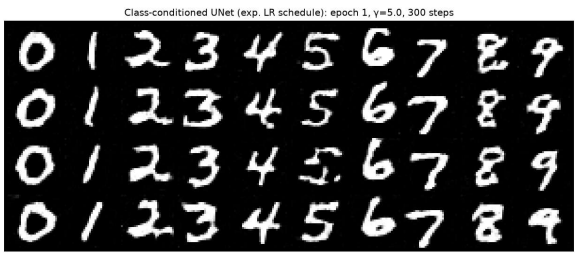
Two more FCBlocks take a one-hot class vector c, and the conditioning becomes `unflatten = c1·unflatten + t1` and `up1 = c2·up1 + t2`. With probability 0.1 the class vector is zeroed during training, so the same network also learns the unconditional flow. That is what enables classifier-free guidance at sampling time.

Same optimizer and schedule as 2.2 (batch 64, D = 64, Adam 1e-2 → 1e-3, 10 epochs). Loss fell from 0.159 to 0.085.

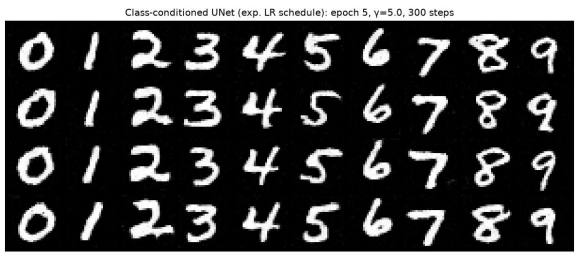


training loss, whole run

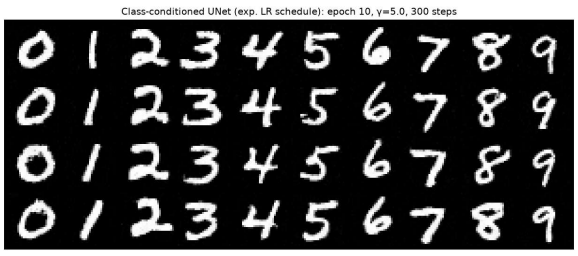
4 SAMPLES OF EACH DIGIT, Γ = 5, 300 STEPS, FIXED SEED



epoch 1



epoch 5

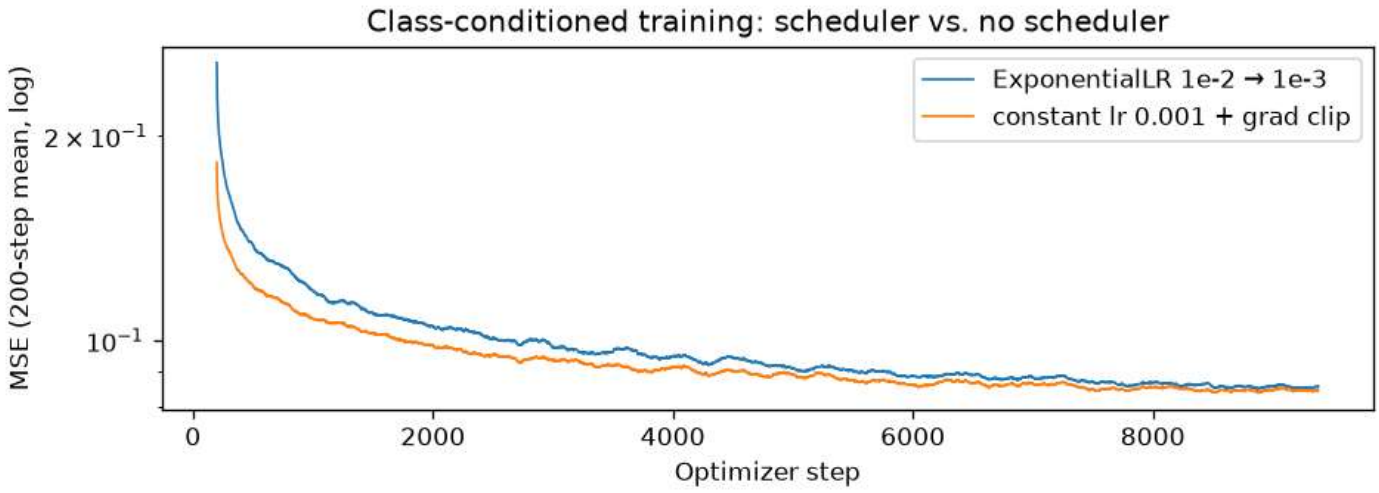


epoch 10

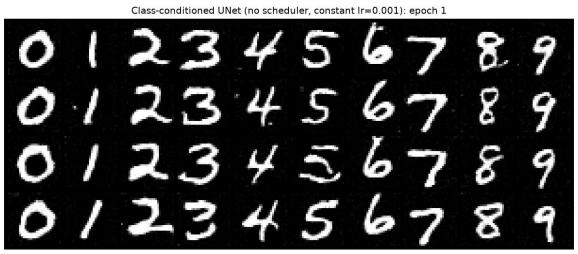
Conditioning plus guidance helps a lot. Even after one epoch every column is the right digit. A few are rough (the 8s and 9s, a broken 5), and by epoch 10 they're all clean, thick, well-formed and still varied in slant and style within each column.

B · 2.6 Getting Rid of the Learning-Rate Scheduler

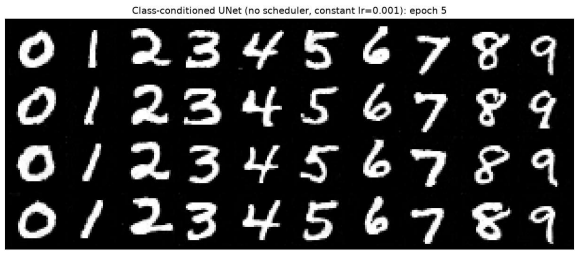
The exponential schedule starts at 1e-2 (fast progress) and ends at 1e-3 (small, stable updates). Without it, a constant 1e-2 would keep taking big, noisy steps all the way to the end. To compensate I used a **constant lr of 1e-3** (the value the schedule decays to) and added **gradient-norm clipping at 1.0** to guard against occasional spikes. Everything else is identical.



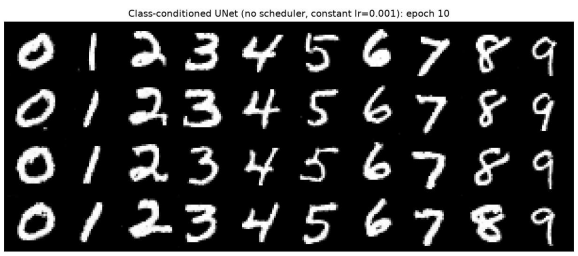
smoothed loss: scheduled vs. constant lr + clipping



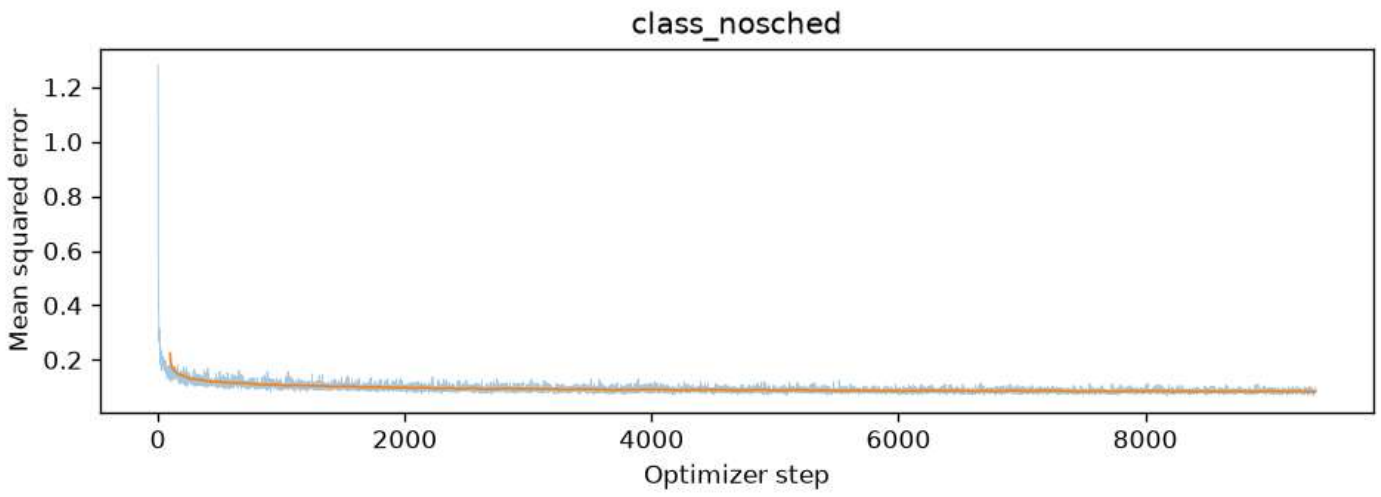
no scheduler, epoch 1



epoch 5



epoch 10



no-scheduler training loss

Result

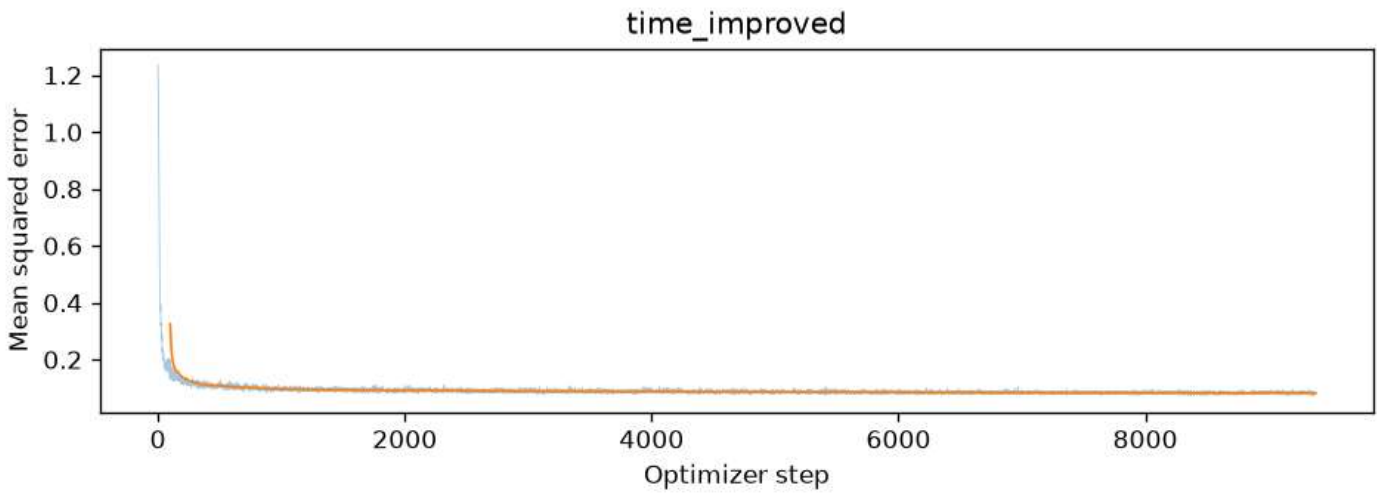
- Final-epoch mean loss is **0.0844 without the scheduler vs. 0.0854 with it**, and the epoch-10 samples look just as good: clean digits in every column.
- The constant-lr run is even ahead early on. Adam already scales each parameter's step, so 1e-3 is plenty for this small model, and 1e-2 mostly buys speed in the first few hundred steps at the cost of noise. Clipping keeps the rare large-gradient batches from undoing progress.

B · BELLS & WHISTLES

A Better Time-Conditioned-Only UNet

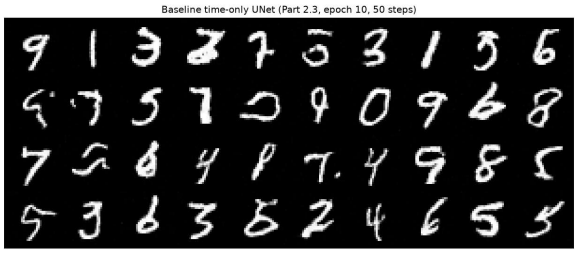
The 2.3 model is the weakest one here. I kept the same flow-matching objective (Algorithm B.1) and the same 50-step Euler sampler, and changed the model and the training:

	BASELINE (2.3)	IMPROVED
time input	raw scalar t → FCBlock	sinusoidal embedding (256-d) → MLP
conditioning	multiply, 2 places	**FiLM** (scale *and* shift, zero-init) at all 6 encoder/decoder stages
width	D = 64	D = 128 (8.55 M params)
optimizer	Adam 1e-2, exp. decay, 10 epochs, batch 64	AdamW 2e-3, 500-step warmup + cosine decay, 20 epochs, batch 128, grad clip 1.0
weights used for sampling	last	**EMA** of weights (decay 0.999)

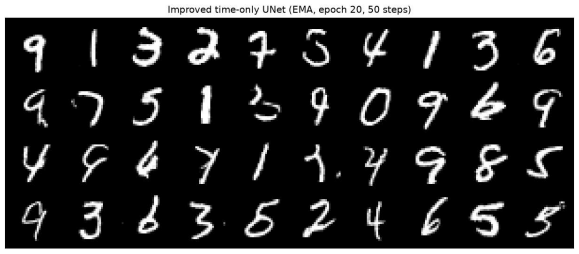


improved model training loss (final epoch 0.0828 vs. 0.0908 for the baseline)

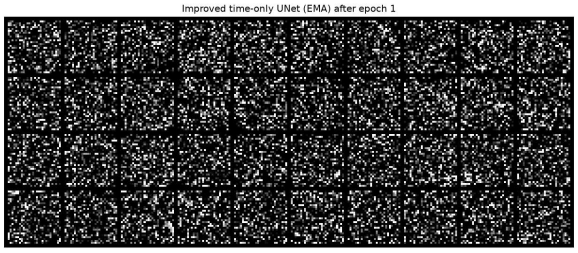
SAME NOISE SEED, SAME 50 EULER STEPS



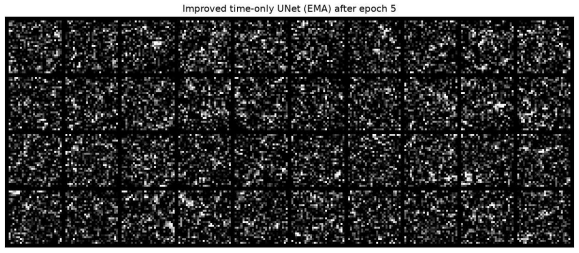
baseline (2.3, epoch 10)



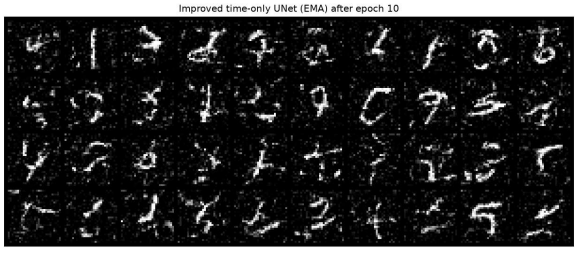
improved (EMA, epoch 20)



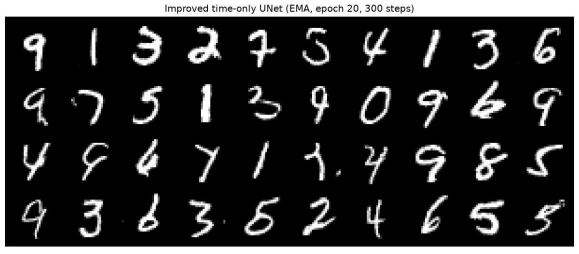
improved, epoch 1



epoch 5



epoch 10



epoch 20, 300 steps

To measure "better" instead of eyeballing it, I trained a small CNN classifier on MNIST (98.6% test accuracy) and ran it on 1000 samples from each model. A good sample is one the classifier is confident about; class entropy checks that all ten digits still appear (the maximum is ln 10 = 2.303).

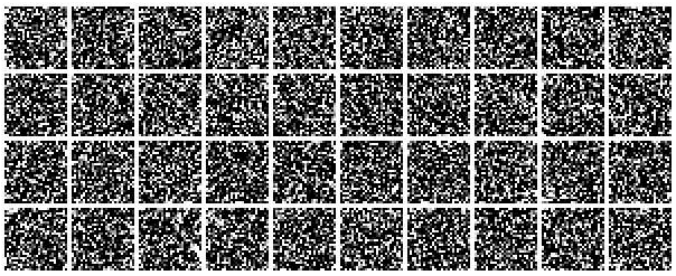
Sample quality judged by an MNIST classifier, 1000 samples each.

MODEL	MEAN CONFIDENCE	CONFIDENCE > 0.9	CLASS ENTROPY
baseline (2.3), 50 steps	0.920	77.6%	2.285
improved (EMA), 50 steps	0.955	87.3%	2.299
improved (EMA), 300 steps	**0.958**	**87.9%**	**2.299**

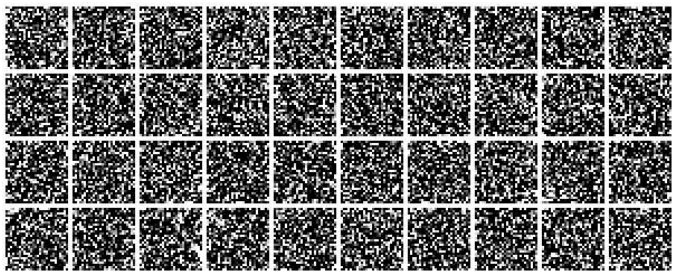
Takeaways

- The share of ambiguous samples (confidence ≤ 0.9) drops from **22% to 13%**, and diversity goes slightly *up*, so it isn't cheating by drawing only easy digits. Visually, the baseline's squiggles and stray marks are mostly gone: same noise, same layout, cleaner strokes.
- FiLM matters most. Time can now shift features as well as scale them, at every resolution. EMA smooths out the last bit of optimizer noise.
- More Euler steps (300 vs. 50) barely help. The remaining errors come from the model, not from integration error.

Extra: watching the samples form



class-conditioned (2.6), 300 steps



improved time-only, 50 steps

Both animations show the same thing as Part A's trajectories: the rough digit shape (low frequencies) is decided early, and the sharp strokes only appear near t = 1.